AI Development and Usage Report

Mini AI Business Intelligence Assistant

Project	Mini AI Business Intelligence Assistant	**Repository**	GitHub project
Stack	Python · Flask · SQLite · HTML/CSS/JavaScript · Gemini API integration	**Purpose**	Assignment development record

1. Project Overview

The application lets users ask natural-language questions about sales transactions through a browser interface. A Flask API receives each question, an interpreter maps it to a supported analytics intent, predefined read-only SQL queries retrieve data from SQLite, and the application returns an answer with supporting figures.

Supported analytics: total revenue; top revenue-generating product; monthly revenue trend; top-performing region; and revenue comparison between two months.

2. Technology and AI Tools

Technology / tool	Purpose
Python and Flask	Application logic and HTTP API.
SQLite	Local relational database for sales transactions.
HTML, CSS, JavaScript	Browser interface.
Google Gemini API / google-genai	Added for natural-language classification into a restricted intent schema; live response not yet verified.
python-dotenv	Loads local environment configuration.
Python unittest	Automated tests.
Git and GitHub	Version control and source-code hosting.
ChatGPT	Planning, code scaffolding, debugging guidance, test guidance and documentation drafting.

3. Architecture

Browser UI → Flask API → Question interpreter → Validated intent → Fixed SQL query → SQLite → Structured answer and supporting figures → Browser UI.

The application exposes / for the interface, POST /api/ask for questions, and /api/health for status and loaded-record count. The SQL helper permits SELECT statements only. User text is not inserted directly into SQL.

4. Data Management and Calculation

The source sales records were supplied as a PDF table and extracted into data/sales_data.csv. The application imports valid rows into SQLite. Dates and numeric fields are parsed; invalid or incomplete rows are skipped; transaction_id is the primary key.

Revenue = quantity × unit_price – **discount_amount.** Only Completed transactions with quantity greater than zero are included. Returned and zero-quantity transactions are excluded. The PDF contains 51 records; TXN0051 has a blank unit price and is skipped, leaving 50 valid imported records.

5. AI Integration and Safety

Gemini integration was added to classify questions into restricted intents: total, product, monthly, region, compare, or unsupported. Month values are validated against January–June. Gemini is not asked to generate SQL; a validated intent selects a predefined SQL template.

Live verification status: Gemini returned HTTP 503 (UNAVAILABLE) with a high-demand message during testing. The application then used its keyword-based fallback, which was verified to return results quickly. A successful live Gemini response has not yet been confirmed.

The API key is stored in a local .env file excluded from Git using .gitignore. Unsupported questions return an error rather than a guessed answer.

6. Testing

Automated tests were run with Gemini disabled to avoid dependence on external API availability:

```
GEMINI_MODE=false python -m unittest discover -s tests -v
```

Test case	Observed result
Total revenue	Passed
Top product	Passed
Monthly revenue	Passed
Top region	Passed
Invalid question	Passed
Month comparison edge case	Passed

7. Development Challenges and Debugging

- The source data was a PDF table rather than a ready-to-import CSV, so records were extracted into CSV format.
- The final source record had a blank unit price; import validation skipped it instead of calculating misleading revenue.
- The Python virtual environment and dependencies were configured to run the Flask application and tests.
- Gemini returned HTTP 503 during live testing; the keyword fallback was retained so supported questions could still be answered.
- Tests were run with Gemini disabled to keep them deterministic and independent of external service availability.

8. Current Limitations

- Keyword fallback recognizes a limited range of question phrasings.
- Month comparison is limited to January–June 2026.
- Live Gemini availability depends on the external service; successful live operation remains unverified.
- No user authentication; intended for local demonstration, not production deployment.

9. Future Improvements

Add bounded API timeouts and controlled retries; verify successful Gemini classification; test malformed model responses; add more analytics, wider date ranges, charts, authentication, logging and production deployment.

10. Repository

https://github.com/sreeja-malloba/mini-ai-business-intelligence-assistant

Accuracy note: This report reflects implementation and test results observed during development. Live Gemini operation was not successfully verified because the service returned HTTP 503.